

Механизмы обработки исключений и многопоточность в C++



Санкт-Петербург
2025

Елена Сергеева

 srgv.lena@gmail.com

План занятия



1. Механизмы обработки исключений в C++



2. Многопоточность и примитивы синхронизации в C++



3. Мастер-класс по отладке с GDB



[Репозиторий](#)

с примерами кода и практическими заданиями, конспектами

Рекомендуемая литература

К. Владимиров, Курс лекций по C++

Э. Уильямс, Параллельное программирование на C++ в действии

cppreference.com

- [исключения](#)
- [многопоточность](#)



Механизмы обработки исключений

```
$ git checkout part1_exceptions  
$ git branch  
main  
* part1_exceptions  
part2_multithreading  
part3_debugging
```

От кодов ошибок к исключениям

Ошибка (Error)

нарушение условий корректной работы программы, которое может быть обнаружено **во время выполнения** или компиляции

Обработка ошибок (Error Handling)

процесс обнаружения, передачи информации и восстановления после ошибок в программе

От кодов ошибок к исключениям

Основные подходы к обработке ошибок:

- **Коды возврата** (return codes) – функция возвращает специальное значение, сигнализирующее об ошибке (C-style)
- **Глобальные переменные** (errno в C) – информация об ошибке сохраняется в глобальной переменной
- **Альтернативы кодам возврата** в C++17/C++23
`std::optional<T>`, `std::variant<T, ErrCode>`, `std::expected<T, Err>`
- **Исключения** (exceptions) - специальный механизм передачи и обработки ошибок

Подходы к обработке ошибок на Си

Коды возврата (Return Codes)

Функция возвращает специальное значение, указывающее на успех или тип ошибки.

Преимущества:

- ✓ Простота реализации
- ✓ Явная обработка ошибок
- ✓ Совместимость с Си
- ✓ Предсказуемая производительность

Недостатки:

- ✗ Легко проигнорировать ошибку
- ✗ Смешивание логики с обработкой ошибок
- ✗ Усложнение при множественных ошибках
- ✗ Проблемы с функциями, которые должны возвращать значение

Подходы к обработке ошибок на Си

Глобальные переменные
(`errno`)

`errno` – это глобальная переменная (макрос в современных реализациях), которая устанавливается библиотечными функциями для указания типа произошедшей ошибки.

Преимущества:

- ✓ **Стандартизованность:** Определен в стандарте C (ISO C90) и POSIX
- ✓ **Универсальность:** Используется множеством системных и библиотечных функций
- ✓ **Детальная информация:** Предоставляет специфические коды ошибок (ENOENT, EACCES, и т.д.)
- ✓ **Потокобезопасность (Thread-safety):** В современных реализациях каждый поток имеет свой `errno`
- ✓ **Обратная совместимость:** Поддержка legacy кода
- ✓ **Низкие накладные расходы:** Простое присваивание значения
- ✓ **Интеграция с системными вызовами:** Прямая связь с операционной системой

Подходы к обработке ошибок на Си

Глобальные переменные
(`errno`)

Проблемы подхода с `errno`:

- ✗ Может быть перезаписана последующими вызовами - требует немедленной проверки
- ✗ Требуется проверки после каждого вызова функции
- ✗ Глобальное состояние может приводить к неожиданному поведению
- ✗ Не все функции устанавливают `errno` корректно
- ✗ Необходимость сброса `errno` перед вызовом функций
- ✗ Не `thread-safe` в старых реализациях (решено в современных системах)

Посмотрим примеры кода
обработки ошибок в стиле Си

`src/error_handling`

└─ `error_codes.c`

└─ `errno_demo.c`

Исключения (exceptions)

Событие, которое возникает во время выполнения программы и нарушает нормальный поток выполнения инструкций. Исключение распространяется “наверх” по стеку вызовов до тех пор пока не будет поймано.

Основная идея: Разделение основной логики программы и кода обработки ошибок.



Ключевые характеристики:

- Исключение **нельзя проигнорировать**
- Исключение **автоматически распространяется** по стеку вызовов
- **Автоматическая очистка ресурсов** при исключениях
- Исключение **прерывает текущий поток выполнения**

```

1 // C-подход: логика смешана с обработкой ошибок
2 int process_file(const char* filename) {
3     FILE* file = fopen(filename, "r");
4     if (!file) return ERROR_FILE;           // ← обработка ошибки
5
6     char* buffer = malloc(1024);
7     if (!buffer) {                          // ← обработка ошибки
8         fclose(file);                       // ← очистка
9         return ERROR_MEMORY;               // ← обработка ошибки
10    }
11
12    size_t bytes = fread(buffer, 1, 1024, file);
13    if (bytes == 0 && ferror(file)) {        // ← обработка ошибки
14        free(buffer);                       // ← очистка
15        fclose(file);                     // ← очистка
16        return ERROR_READ;                // ← обработка ошибки
17    }
18
19    // Основная логика занимает 20% кода!
20    process_data(buffer, bytes);
21    free(buffer);                          // ← очистка
22    fclose(file);                         // ← очистка
23    return SUCCESS;
24 }

```

```

1 // C++ подход: чистая логика
2 void process_file(const std::string& filename) {
3     std::ifstream file(filename);
4     if (!file) throw std::runtime_error("Cannot open file");
5
6     std::vector<char> buffer(1024);
7     file.read(buffer.data(), buffer.size());
8
9     if (file.bad()) throw std::runtime_error("Read error");
10
11    // Основная логика занимает 80% кода!
12    process_data(buffer.data(), file.gcount());
13
14    // Автоматическая очистка благодаря деструкторам!
15 }

```



Выбрасывание исключения: `throw`

Выбросить можно исключение любого типа

✓ Хорошая практика –выбрасывать объекты класса `std::exception` или его наследников

Блок защищенного кода: `try { ... }`

Перехват и обработка исключения:

`catch (...) { ... }`

```
1 try {  
2     // Код, который может выбросить исключение  
3     process_file("data.dat");  
4  
5     // Если исключение выброшено, выполнение прерывается  
6     // и управление передается в блок catch  
7  
8 } catch (const std::exception& e) {  
9     // Обработка исключения  
10    std::cerr << "Error: " << e.what() << std::endl;  
11 }
```

Множественные catch-блоки

Порядок catch блоков важен!

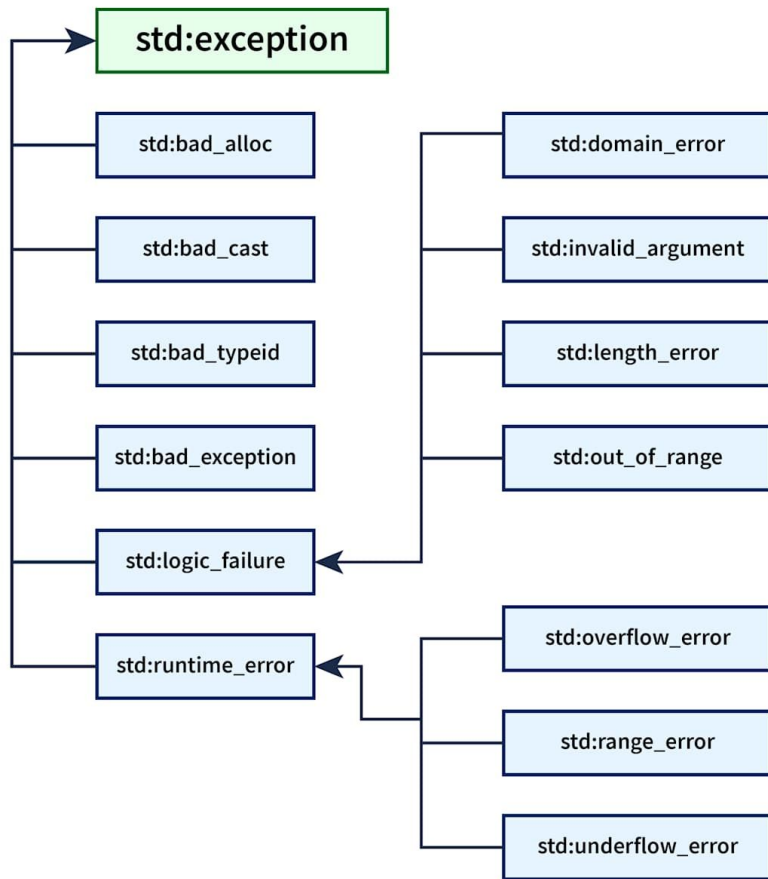
от специфичных исключений к
общим

Сначала наследники, потом
базового типа

```
1 try {
2     risky_operation();
3 }
4 catch (const std::bad_alloc& e) {
5     // Специфическая обработка нехватки памяти
6     std::cerr << "Out of memory: " << e.what() << std::endl;
7     cleanup_memory();
8 }
9 catch (const std::runtime_error& e) {
10    // Обработка runtime ошибок
11    std::cerr << "Runtime error: " << e.what() << std::endl;
12    log_error(e.what());
13 }
14 catch (const std::exception& e) {
15    // Обработка всех остальных std::exception
16    std::cerr << "General error: " << e.what() << std::endl;
17 }
18 catch (...) {
19    // Перехват всех остальных исключений
20    std::cerr << "Unknown error" << std::endl;
21    throw; // Пробрасываем дальше
22 }
```

Иерархия стандартных исключений

[std::exception](#)



Время практики

Давайте перепишем C-style код с использованием исключений (вместо кодов возврата)

```
src/task1_1.cpp  
src/task1_1.hpp
```


RAII (Resource Acquisition is Initialization)

RAII - это идиома программирования, которая связывает время жизни ресурса со временем жизни объекта.

Основные принципы:

- Ресурс приобретается в конструкторе объекта
- Ресурс освобождается в деструкторе объекта
- Время жизни ресурса = время жизни объекта
- Автоматическое управление - программист не думает об освобождении

Глубокое погружение в исключения

Раскрутка стека (Stack Unwinding)

Stack Unwinding

- поиск ближайшего подходящего `catch block` вверх по стеку вызовов
- вызов деструкторов для всех автоматических объектов, созданных с момента входа в блок `try` до точки возникновения исключения.
RAII позволяет корректно управлять ресурсами даже в исключительных ситуациях.
- если `catch block` не найден будет вызван `std::terminate()`

Глубокое погружение в исключения

Раскрутка стека (Stack Unwinding)

Гарантия вызова деструкторов:

- Все полностью сконструированные автоматические объекты будут уничтожены
- Деструкторы вызываются в **обратном порядке** конструирования
- Для частично сконструированных объектов деструкторы не будут вызваны

Производительность Stack Unwinding:

- Stack unwinding может быть дорогой операцией
 - Требуется поиска обработчиков исключений
 - Вызов множественных деструкторов
- В критичных к производительности участках стоит минимизировать исключения

Исключения в деструкторах

Если при раскрутке стека возникнет **еще одно исключение**, то будет вызван `std::terminate()`

По этой причине выброс исключения за пределы деструктора может привести к **аварийному завершению программы**

По умолчанию все деструкторы в C++ помечены как `noexcept`

Время познакомиться с Stack Unwinding на практике

(и немного с отладкой из IDE)

```
src/error_handling/stack_unwinding.cpp
```

Гарантии безопасности относительно исключений (Exception Safety)

No Exception Safety (Нет гарантий): при исключении могут произойти утечки ресурсов и объект может остаться в некорректном состоянии.

Basic Guarantee (Базовые гарантии): при исключении не происходит утечек ресурсов, и все объекты остаются в валидном состоянии, но состояние может быть изменено.

Strong Guarantee (Строгие гарантии): при исключении состояние объекта остается точно таким же, как до начала операции (commit or rollback semantics).

No-Throw Guarantee (Гарантия отсутствия исключений): операция гарантированно не выбросит исключения. Помечаются спецификатором `noexcept`

Концепция безопасного периметра

Безопасный периметр в C++ основывается на создании концентрических зон безопасности, где каждый уровень предоставляет определенные гарантии при возникновении исключений. При проектировании системы с использованием концепции безопасного периметра рекомендуется следующая архитектура:

1. **Ядро системы** - код с строгими гарантиями безопасности, использующий RAII и умные указатели
2. **Промежуточные слои** - компоненты с базовыми гарантиями безопасности
3. **Границы взаимодействия** - слои трансляции исключений для взаимодействия с внешними системами
4. **Внешние интерфейсы** - C-совместимые интерфейсы без исключений

Рекомендации

- **Используйте RAII везде, где возможно.** Каждый ресурс должен управляться объектом RAII, обеспечивающим автоматическое освобождение.
- **Минимизируйте области с отсутствием гарантий безопасности.** Такой код должен быть изолирован и тщательно протестирован.
- **Создавайте четкие границы исключений.** Все исключения должны обрабатываться на соответствующих уровнях архитектуры.
- **Предпочитайте композицию наследованию.** Это упрощает управление ресурсами и обеспечение гарантий безопасности.

Еще немного практики...

и перерыв :)

Познакомимся с **Copy&Swap** идиомой ,
которая обеспечивает строгие гарантии
безопасности относительно исключений

`src/error_handling/safe_container.cpp`

Рефлексия по части 1

- Рассмотрели подходы в обработке ошибок в программах на Си/С++

Обсудили их достоинства и недостатки

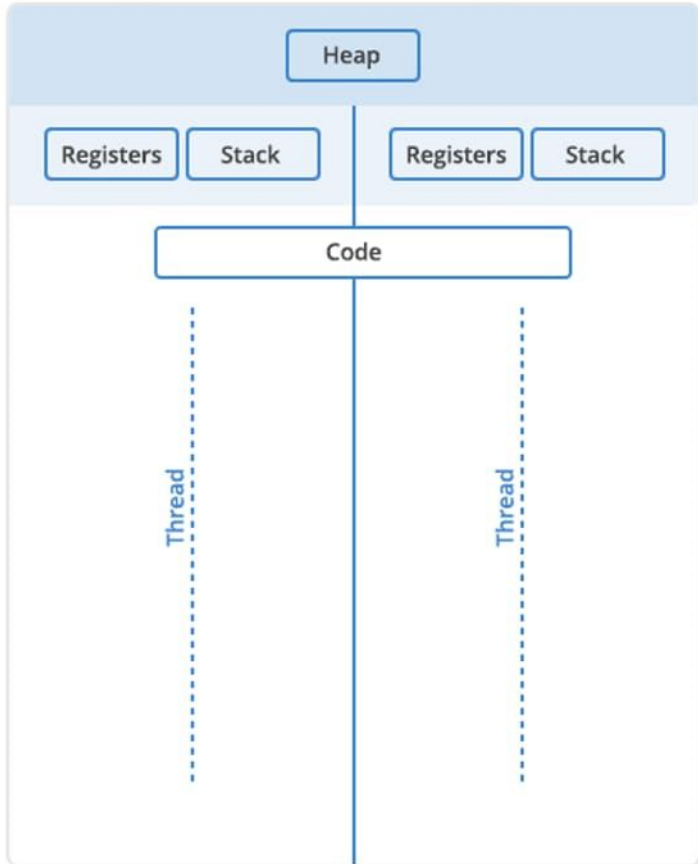
- Изучили как работать с исключениями на С++

- Рассмотрели гарантии безопасности

Многопоточность и примитивы синхронизации

```
$ git checkout part2_multithreading
$ git branch
  main
  part1_exceptions
* part2_multithreading
  part3_debugging
```

Multi Threaded



Поток (thread) – независимый путь выполнения инструкций внутри процесса, минимальная единица планирования ОС. Потоки позволяют выполнять операции параллельно, но при этом есть риск **гонок данных (data races)**.



Ключевые характеристики:

- **Общее адресное пространство:** все потоки процесса видят одну память
- **Собственный стек:** каждый поток имеет свой стек вызовов
- **Общие глобальные переменные**
- **Переключение контекста:** ОС управляет выполнением потоков

Потоки в C++

`std::thread` – базовый механизм запуска параллельных потоков (начиная с C++11). Позволяет запускать функции, классы, лямбда-выражения в отдельных потоках.

- ✓ **Type safety:** компилятор проверяет типы параметров
- ✓ **RAII:** автоматическое управление ресурсами
- ✓ **Exception safety:** исключения корректно передаются
- ✓ **Платформонезависимость**

⚠ **Забывтый `join()`** - вызывает `std::terminate()`

⚠ **Передача ссылок** - требует `std::ref()`

⚠ **Исключения** - могут прервать `join()`

`std::jthread` (joining thread) – усовершенствованная версия `std::thread`, решающая основные проблемы безопасности и удобства использования.

+ Автоматическое присоединение (Auto-joining)

+ Встроенная поддержка отмены через `std::stop_token`

+ Улучшенная RAII совместимость

Характеристика	std::thread	std::jthread
Стандарт C++	C++11	C++20
Автоматическое присоединение	✗ Требуется явный join()/detach()	✓ Автоматически в деструкторе
Поведение при забытом join()	💣 std::terminate	✓ Безопасное завершение
Встроенная отмена	✗ Только через пользовательские флаги	✓ std::stop_token
RAII совместимость	⚠ Требуется осторожности	✓ Естественная интеграция
Размер объекта	~8 байт	~16 байт
Производительность	Минимальные накладные расходы	Небольшие дополнительные расходы

Посмотрим на примерах кода основы работы с потоками

```
src/multithreading
```

```
|— std_thread_demo.cpp
```

```
|— std_jthread_demo.cpp
```

Гонка данных

Data race



Ошибка параллельного программирования, при которой работа многопоточной системы зависит от **порядка выполнения операций**

- ❖ приводит к несогласованности данных или нарушению инварианта
- ❖ проявляется в произвольные моменты времени
- ❖ трудно воспроизводима
- ❖ вероятность возникновения повышается с повышением нагрузки на систему

Атомарные операции – неделимые операции. Выполняются полностью или не выполняются совсем. **Частичное выполнение невозможно.**

```
1 std::atomic<int> atomic_int{0};
2 // Загрузка значения
3 int value = atomic_int.load();
4 // Сохранение значения
5 atomic_int.store(42);
6 // Обмен значения
7 int old_value = atomic_int.exchange(new_value);
8 // Compare-and-swap
9 int expected = 10;
10 bool success = atomic_int.compare_exchange_weak(expected, 20);
11 // Арифметические операции
12 atomic_int.fetch_add(5);           // возвращает старое значение
13 atomic_int.fetch_sub(3);
14 atomic_int += 2;                   // возвращает новое значение
15 ++atomic_int;
```

`std::atomic<T>` (начиная C++11)

Здесь будет практика...
Воспроизведем гонку данных и
исправим на атомарный инкремент

`src/task2_1.cpp`

А потом обед :)

Зачем синхронизация в многопоточной программе ?

Обеспечение корректной
работы с разделяемыми
ресурсами

Синхронизация
(упорядочивание) действий,
выполняемых разными
потоками

Зачем синхронизация в многопоточной программе ?

Обеспечение корректной
работы с разделяемыми
ресурсами

Синхронизация
(упорядочивание) действий,
выполняемых разными
потоками

Взаимное исключение

Mutex (Mutual Exclusion)

Критическая секция – участок кода, который может выполняться одновременно только одним потоком.

Примитив синхронизации для обеспечения критических секций: дает гарантии, что в критической секции находится только один поток

Состояния mutex

- ❖ занят (**locked**)
- ❖ свободен (**unlocked**)

Операции над mutex

- ❖ захватить (**lock**, acquire)
- ❖ освободить (**unlock**, release)



**⚠ Использование мьютекса
ведет к сериализации**

**Примитив
синхронизации
(mutex)**

ЛОГИЧЕСКАЯ СВЯЗЬ

**разделяемый ресурс
(структура данных)**



Типы мьютексов

- ❖ **нерекурсивные**
`std::mutex`
- ❖ **рекурсивные**: можно захватить в одном потоке несколько раз
`std::recursive_mutex`
- ❖ **разделяемые по чтению и записи**
(shared, read-write)
`std::shared_mutex`
- ❖ **с таймаутом**
`std::timed_mutex`

RAII (Resource Acquisition Is Initialization)

Notes

`std::mutex` is usually not accessed directly: `std::unique_lock`, `std::lock_guard`, or `std::scoped_lock` (since C++17) manage locking in a more exception-safe manner.

RAII обёртки над мьютексом в C++

Хорошая практика

- `std::lock_guard`
простая RAII обёртка
- `std::unique_lock`
гибкая блокировка, множественный захват/освобождение
- `std::scoped_lock` (C++17)
для захвата нескольких мьютексов. Позволяет заблокировать несколько мьютексов и избежать взаимной блокировки (deadlock)
- `std::shared_lock` (C++14):
для мьютекса разделяемого на чтение и запись

Пример использования RAII оберток мьютекса

Грубая блокировка списка

```
class safe_list{
private :
    std::list<int> _list ;
    std::mutex _mutex;
public :
    void safe_add( int new_value){
        // захват примитива с использованием замка
        std::lock_guard<std::mutex> guard(_mutex);
        _list.push_back(new_value );
    }

    bool contains ( int value_to_find){
        std::lock_guard<std::mutex> guard(_mutex);
        return std::find (some_list.begin(),
                           some_list.end() ,
                           value_to_find)  $\neq$  _list.end ();
    }
}
```

Попрактикуемся в ИСПОЛЬЗОВАНИИ МЬЮТЕКСОВ

`src/task2_2.cpp`

`src/multithreading/std_mutex_example.cpp`

Зачем синхронизация в многопоточной программе ?

Обеспечение корректной
работы с разделяемыми
ресурсами

Синхронизация
(упорядочивание) действий,
выполняемых разными
потоками

Условные переменные

- ❖ Над условными переменными определены 2 парные операции:
 - **Wait** (ожидать события)
 - **Notify** (сообщить о событии)
- ❖ Условные переменные используются вместе с мьютексом
- ❖ Ассоциированы с каким-либо событием (условием)
- ❖ Один или несколько потоков могут ждать когда произойдёт событие (выполнится условие)

[std :: condition_variable](#)

[std :: condition_variable_any](#)

Использование условных переменных

задача
“производитель потребитель”

```
mutex mut;
queue<data_chunk> data_queue ;
condition_variable data_cond;
void Producer(){
    while(true){
        data_chunk const data=produce_item();
        lock_guard<mutex> lk (mut);
        data_queue.push(data);
        data_cond.notify_one();
    }
}
void Consumer(){
    while(true){
        unique_lock<mutex> lk(mut);
        data_cond.wait(lk, []{ return !data_queue.empty();});
        data_chunk data=data_queue.front();
        data_queue.pop();
        lk.unlock();
        process(data);
    }
}
```

Что происходит при вызове Wait ?

- 1. мьютекс захватывается
- 2. проверяется условие
- 3. если условие не выполнилось, то мьютекс освобождается, поток переходит в ожидание
- 4. приходит уведомление – просыпается поток, захватывает мьютекс, проверяется условие
- и если оно не выполнилось возвращаемся на 3

Мьютекс захватывается и освобождается → нужно использовать `unique_lock`



Спонтанные пробуждения (Spurious wakeup) → проверка условия непредсказуемое количество раз → проверка условия должна быть без побочных эффектов

И еще одно практическое
задание:
сделать очередь ограниченной

```
src/multithreading/cond_var.cpp
```

Барьеры

Барьеры позволяют нескольким потокам координировать выполнение так, чтобы ни один из них не продолжил работу, пока все остальные не достигнут определённой точки



- Барьер может быть одноразовым или многократным (циклическим);
- Используется в алгоритмах с фазовой обработкой
- Может быть дорогим по времени ожидания, если отдельные потоки сильно задерживаются.

Барьеры (C++20)

[std::barrier](#)

[std::latch](#)

```
1 #include <barrier>
2 #include <thread>
3 #include <vector>
4 #include <iostream>
5
6 void work(std::barrier<> &b, int id) {
7     std::cout << id << " before barrier" << std::endl;
8     // and some work here
9     b.arrive_and_wait();
10    std::cout << id << " after barrier" << std::endl;
11 }
12
13 int main() {
14     std::barrier b(4); // for 4 threads
15     std::vector<std::jthread> threads;
16     for(int i = 0; i < 4; ++i)
17         threads.emplace_back(work, std::ref(b), i);
18 }
```

Высокоуровневое управление потоками в C++

`std::async` - это высокоуровневая функция, которая позволяет запускать задачи асинхронно, автоматически управляя созданием потоков и возвращая **`std::future`** для получения результата.

`std::future` предоставляет интерфейс для получения результата асинхронной операции.

- ✓ **Автоматическое управление потоками** - не нужно создавать `std::thread` вручную
- ✓ **Простая передача результатов** - через `std::future`
- ✓ **Гибкие политики запуска** - `sync/deferred/auto`
- ✓ **Exception-safe** - исключения передаются через `future`

Рефлексия по части 2

- Изучили основы работы с потоками в C++
- Познакомились с основными примитивами синхронизации
- Познакомились с высокоуровневым подходом к управлению потоками

Мастер-класс по отладке с GDB

```
$ git checkout part3_debugging
$ git branch
  main
  part1_exceptions
  part2_multithreading
* part3_debugging
```

Рефлексия по части 3

- Изучили основы отладки с использованием GDB
- Познакомились с отладкой исключений средствами GDB
- Познакомились с отладкой многопоточных программ в GDB

Выявлены и решены проблемы:

1. **Buffer overflow в `std::vector`** → исправлено условие цикла
2. **Memory leak при исключениях** → заменён raw pointer на `std::unique_ptr`
3. **Race condition** → добавлен `std::mutex` с `std::lock_guard`
4. **Deadlock** → использованы `std::lock()` или `std::scoped_lock()` для одновременной блокировки